

Mini RFC: Probe Timeout

May 19, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

A Probe Timeout (PTO) triggers the sending of one or two probe datagrams when ack-eliciting packets are not acknowledged within the expected period of time or the server may not have validated the client's address. A PTO enables a connection to recover from loss of tail packets or acknowledgments.

As with loss detection, the PTO is per packet number space. That is, a PTO value is computed per packet number space.

A PTO timer expiration event does not indicate packet loss and MUST NOT cause prior unacknowledged packets to be marked as lost. When an acknowledgment is received that newly acknowledges packets, loss detection proceeds as dictated by the packet and time threshold mechanisms, as described in *mini RFC: Reliability*.

An endpoint could send a packet on a PTO timer expiration even if it would cause `bytes_in_flight` to be larger than the congestion window.

The PTO algorithm used in QUIC implements the reliability functions of Tail Loss Probe [RFC8985], RTO [RFC5681], and F-RTO algorithms for TCP [RFC5682]. The timeout computation is based on TCP's RTO period [RFC6298].

People keep improving TCP but the compatibility with old implementations turns to be a hard issue.

2 PTO Details

2.1 Computing PTO

When an ack-eliciting packet is transmitted, the sender schedules a timer for the PTO period as follows:

$$\text{PTO} = \text{smoothed_rtt} + \max(4 * \text{rttvar}, \text{kGranularity}) + \text{max_ack_delay}$$

The PTO period is the amount of time that a sender ought to wait for an acknowledgment of a sent packet. This time period includes the estimated network RTT (`smoothed_rtt`), the variation in the estimate ($4 * \text{rttvar}$), and `max_ack_delay`, to account for the maximum time by which a receiver might delay sending an acknowledgment.

When the PTO is armed for Initial or Handshake packet number spaces, the `max_ack_delay` in the PTO period computation is set to 0, since the peer is expected to not delay these packets intentionally.

The PTO period MUST be at least `kgranularity` to avoid the timer expiring immediately.

When ack-eliciting packets in multiple packet number spaces are in flight, the timer MUST be set to the earlier value of the Initial and Handshake packet number spaces.

An endpoint MUST NOT set its PTO timer for the Application Data packet number space until the handshake is confirmed. Doing so prevents the endpoint from retransmitting information in packets when either the peer does not yet have the keys to process them or the endpoint does not yet have the keys to process their acknowledgments. For example, this can happen when a client sends 0-RTT packets to the server; it does so without knowing whether the server will be able to decrypt them. Similarly, this can happen when a server sends 1-RTT packets before confirming that the client has verified the server's certificate and can therefore read these 1-RTT packets.

A sender SHOULD restart its PTO timer every time an ack-eliciting packet is sent or acknowledged, or when Initial or Handshake keys are discarded. This ensures the PTO is always set based on the latest estimate of the RTT and for the correct packet across packet number spaces.

When a PTO timer expires, the PTO backoff MUST be increased, resulting in the PTO period being set to twice its current value. The PTO backoff factor is reset when an acknowledgment is received, except in the following case. A server might take longer to respond to packets during the handshake than otherwise. To protect such a server from repeated client probes, the PTO backoff is not reset at a client that is not yet certain that the server has finished validating the client's address. That is, a client does not reset the PTO backoff factor on receiving acknowledgments in Initial packets.

This exponential reduction in the sender's rate is important because consecutive PTOs might be caused by loss of packets or acknowledgments due to severe congestion. Even when there are ack-eliciting packets in flight

We send PTO packets only when we want to trigger ACK. It's OK if we have sent something as "free" PTO packets.

in multiple packet number spaces, the exponential increase in PTO occurs across all spaces to prevent excess load on the network. For example, a timeout in the Initial packet number space doubles the length of the timeout in the Handshake packet number space.

The total length of time over which consecutive PTOs expire is limited by the idle timeout.

The PTO timer **MUST NOT** be set if a timer is set for time threshold loss detection. A timer that is set for time threshold loss detection will expire earlier than the PTO timer in most cases and is less likely to spuriously retransmit data.

2.2 Sending Probe Packets

When a PTO timer expires, a sender **MUST** send at least one ack-eliciting packet in the packet number space as a probe. An endpoint **MAY** send up to two full-sized datagrams containing ack-eliciting packets to avoid an expensive consecutive PTO expiration due to a single lost datagram or to transmit data from multiple packet number spaces. All probe packets sent on a PTO **MUST** be ack-eliciting.

In addition to sending data in the packet number space for which the timer expired, the sender **SHOULD** send ack-eliciting packets from other packet number spaces with in-flight data, coalescing packets if possible. This is particularly valuable when the server has both Initial and Handshake data in flight or when the client has both Handshake and Application Data in flight because the peer might only have receive keys for one of the two packet number spaces.

If the sender wants to elicit a faster acknowledgment on PTO, it can skip a packet number to eliminate the acknowledgment delay.

An endpoint **SHOULD** include new data in packets that are sent on PTO expiration. Previously sent data **MAY** be sent if no new data can be sent. Implementations **MAY** use alternative strategies for determining the content of probe packets, including sending new or retransmitted data based on the application's priorities.

It is possible the sender has no new or previously sent data to send. As an example, consider the following sequence of events: new application data is sent in a STREAM frame, deemed lost, then retransmitted in a new packet, and then the original transmission is acknowledged. When there is no data to send, the sender **SHOULD** send a PING or other ack-eliciting frame in a single packet, rearming the PTO timer.

Alternatively, instead of sending an ack-eliciting packet, the sender **MAY** mark any packets still in flight as lost. Doing so avoids sending an additional packet but increases the risk that loss is declared too aggressively, resulting in an unnecessary rate reduction by the congestion controller.

Consecutive PTO periods increase exponentially, and as a result, connection recovery latency increases exponentially as packets continue to be dropped in the network. Sending two packets on PTO expiration increases

resilience to packet drops, thus reducing the probability of consecutive PTO events.

When the PTO timer expires multiple times and new data cannot be sent, implementations must choose between sending the same payload every time or sending different payloads. Sending the same payload may be simpler and ensures the highest priority frames arrive first. Sending different payloads each time reduces the chances of spurious retransmission.

2.3 Handshakes and New Paths

Resumed connections over the same network MAY use the previous connection's final smoothed RTT value as the resumed connection's initial RTT. When no previous RTT is available, the initial RTT SHOULD be set to 333 milliseconds. This results in handshakes starting with a PTO of 1 second, as recommended for TCP's initial RTO; see Section 2 of [RFC6298].

A connection MAY use the delay between sending a `PATH_CHALLENGE` and receiving a `PATH_RESPONSE` to set the initial RTT for a new path, but the delay SHOULD NOT be considered an RTT sample.

Related optional features: *Path Validation*.

When the Initial keys and Handshake keys are discarded (see Section 6.4), any Initial packets and Handshake packets can no longer be acknowledged, so they are removed from bytes in flight. When Initial or Handshake keys are discarded, the PTO and loss detection timers MUST be reset, because discarding keys indicates forward progress and the loss detection timer might have been set for a now-discarded packet number space.

Related optional features: *QUIC Handshake*.

Until the server has validated the client's address on the path, the amount of data it can send is limited to three times the amount of data received, as described in *mini RFC: Address Validation*. If no additional data can be sent, the server's PTO timer MUST NOT be armed until datagrams have been received from the client because packets sent on PTO count against the anti-amplification limit.

When the server receives a datagram from the client, the amplification limit is increased and the server resets the PTO timer. If the PTO timer is then set to a time in the past, it is executed immediately. Doing so avoids sending new 1-RTT packets prior to packets critical to the completion of the handshake. In particular, this can happen when 0-RTT is accepted but the server fails to validate the client's address.

Since the server could be blocked until more datagrams are received from the client, it is the client's responsibility to send packets to unblock the server until it is certain that the server has finished its address validation (see *mini RFC: Address Validation* and *mini RFC: Path Validation*). That is, the client MUST set the PTO timer if the client has not received an acknowledgment for any of its Handshake packets and the handshake is not confirmed (see *mini RFC: QUIC Handshake*), even if there are no packets in flight. When the PTO fires, the client MUST send a Handshake packet if it has Handshake keys, otherwise it MUST send an Initial packet in a UDP datagram with a payload of at least 1200 bytes.

3 Packets and Frames

3.1 Packets

There is no new type of packets introduced in this RFC.

3.2 Frames

There is no new type of frames introduced in this RFC.

4 Application Protocol

There is no new application protocol defined in this mini RFC.